



Evolutionary Computation

Assignment 10: Own Method

BENGRICHE Tidiane (ER-2037)
SZCZUBLIŃSKA Joanna Wiktoria (156070)



Contents

1	Introduction	1
2	Pseudocode	2
3	Plots	5
4	Results	7
5	Conclusion	8

1 Introduction

In this assignment, we redesigned our iterated local search (ILS) compared to assignment_6 to improve solution quality and robustness. Instead of a single random start and a simple shake, we now use a multi-start approach and a stronger perturbation.

- Multi-start seeds: build a small pool of starting tours from several well-spread start cities using simple construction rules, plus one random tour; briefly improve each and launch ILS from the best seed. The starting pool includes nearest-neighbor tours, flexible nearest-neighbor, greedy cycle constructions, and k-regret weighted constructions, alongside a random tour.
- Stronger perturbation: cut the tour into four parts, reorder them, sometimes reverse one or two parts, and relocate a few cities to new positions; this shake escapes stalls while preserving useful structure. Previously, the perturbation selected four distinct cut points to form segments A, B, C, and D and reconnected them as A–C–B–D instead of A–B–C–D, removing four edges and creating four new ones while keeping the tour valid.

Together, these changes broaden coverage of the search space and more consistently lead to better scores.

You can see the repository of our assignment on github:

https://github.com/tbengric/EC_laboratory

2 Pseudocode

In this section, we will present the pseudocodes of similarity algorithms use to calculate the similarities. For this assignment, we will code in C++.

Initial Solution Generator (generateInitialSolutions)

```
FUNCTION generateInitialSolutions(dist_matrix, nodes):  
  n  $\leftarrow$  |nodes|  
  start_nodes  $\leftarrow$  [0,  $\lfloor n/3 \rfloor$ ,  $\lfloor 2n/3 \rfloor$ ,  $n - 1$ ] (unique, sorted)  
  seeds  $\leftarrow$  EMPTY_LIST()  
  selected_nodes  $\leftarrow$  SELECT_NODES(n)  
  APPEND(seeds, RANDOM_SOLUTION(selected_nodes))  
  
  FOR s IN start_nodes DO :  
    APPEND(seeds, NEAREST_NEIGHBOR_END(dist_matrix, nodes, s))  
    APPEND(seeds, NEAREST_NEIGHBOR_FLEXIBLE(dist_matrix, nodes,  
s))  
    APPEND(seeds, GREEDY_CYCLE(dist_matrix, nodes, s))  
    APPEND(seeds,  
GREEDY_CYCLE_K_REGRET_WEIGHTED(dist_matrix, nodes, s, k=2,  
 $\alpha=0.5$ ))  
    APPEND(seeds,  
NEAREST_NEIGHBOR_K_REGRET_WEIGHTED(dist_matrix, nodes, s,  
k=2,  $\alpha=0.5$ ))  
  
  RETURN seeds
```

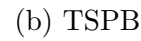

Perturbation (perturbation_strong)

```
FUNCTION perturbation_strong(path):  
  n ← LENGTH(path)  
  IF n < 8 THEN RETURN path  
  cuts ← SELECT_4_DISTINCT(1..n-2) and SORT  
  A ← path[0..cuts[0]-1], B ← path[cuts[0]..cuts[1]-1],  
  C ← path[cuts[1]..cuts[2]-1], D ← path[cuts[2]..n-1]  
  
  IF COIN_FLIP() THEN REVERSE(B)  
  IF COIN_FLIP() THEN REVERSE(C)  
  res ← CONCAT(A, C, B, D)  (reorder as A–C–B–D)  
  
  moves ← MAX(1, ⌊n/50⌋)  (small relocations)  
  FOR k FROM 1 TO moves DO :  
    from ← RANDOM_INT(1, |res|-2)  
    node ← res[from]; DELETE_AT(res, from)  
    to ← RANDOM_INT(1, |res|-1)  
    INSERT_AT(res, to, node)  
  
  RETURN res
```

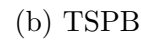
Iterated Local Search (ILS)

```
FUNCTION ILS(dist_matrix, nodes, time_limit_ms):  
  seeds ← generateInitialSolutions(dist_matrix, nodes)  
  best_solution ← NULL, best_score ←  $\infty$   
  local_search_count ← 0  
  
  FOR seed IN seeds DO :  
    improved ← STEEPEST_LOCAL_SEARCH(seed, dist_matrix, "edge",  
nodes)  
    local_search_count ← local_search_count + 1  
    IF improved is invalid THEN CONTINUE  
    score ← COMPUTE_OBJECTIVE(improved, dist_matrix, nodes)  
    IF score < best_score THEN :  
      best_score ← score, best_solution ← improved  
  
  x ← best_solution  
  START_TIMER()  
  WHILE ELAPSED_MS() < time_limit_ms DO :  
    y ← perturbation_strong(x)  
    improved_y ← STEEPEST_LOCAL_SEARCH(y, dist_matrix, "edge",  
nodes)  
    local_search_count ← local_search_count + 1  
    IF improved_y is invalid THEN CONTINUE  
    y_score ← COMPUTE_OBJECTIVE(improved_y, dist_matrix, nodes)  
    IF y_score < best_score THEN :  
      best_score ← y_score, best_solution ← improved_y, x ← improved_y  
  
  RETURN best_solution, best_score, local_search_count
```

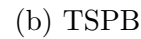
(a) TSPA



(a) TSPA



(a) TSPA



5

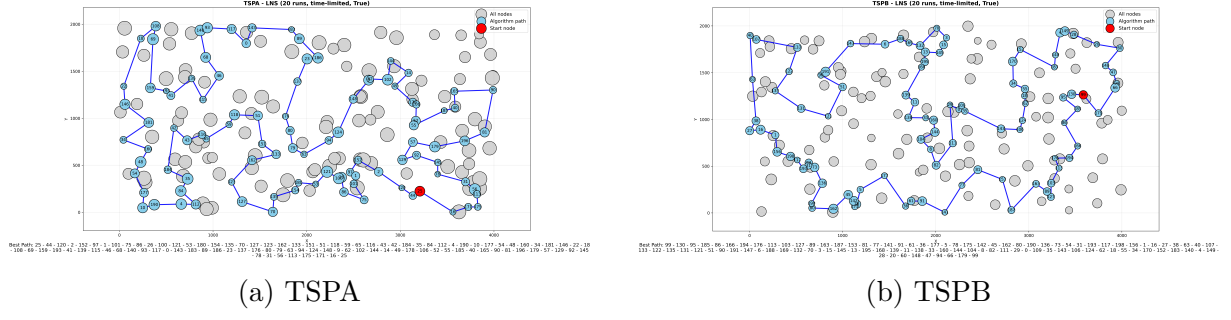


Figure 4: LNS

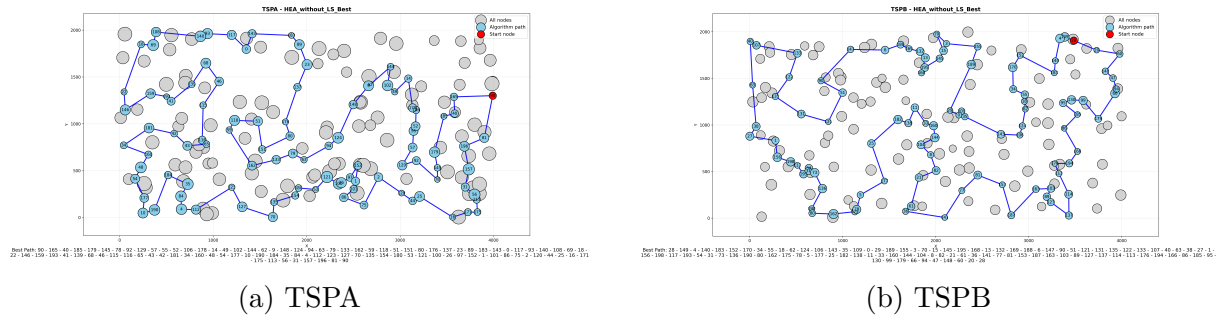


Figure 5: HEA without LS best

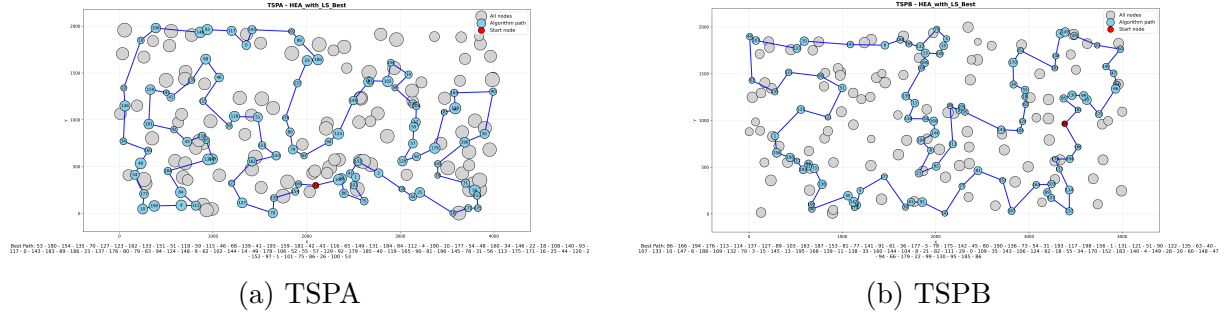


Figure 6: HEA with LS best

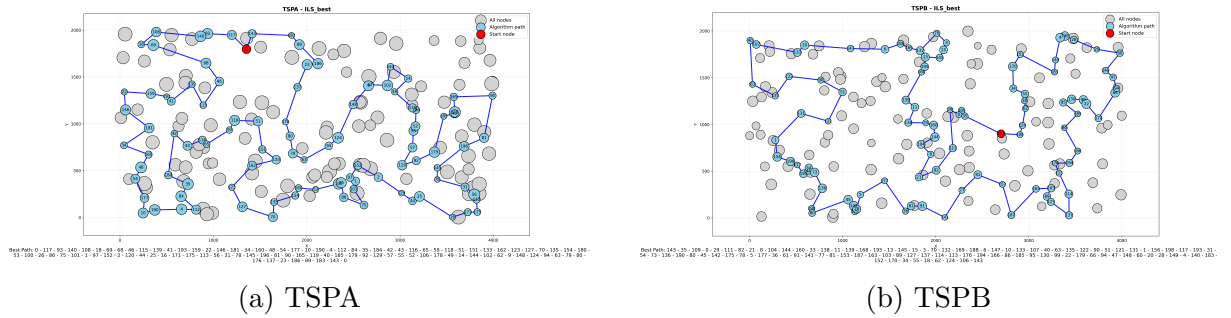


Figure 7: ILS improved

4 Results

Metric	TSPA	TSPB
Average Local Searches per ILS Run	5107.55	5253.45
Min Local Searches	4802	4652
Max Local Searches	5824	5753

Table 1: ILS local search counts for TSPA and TSPB

Metric	TSPA	TSPB
Average Local Searches per ILS improved Run	2483.50	2475.15
Min Local Searches	2150	2300
Max Local Searches	2607	2775

Table 2: ILS improved local search counts for TSPA and TSPB

Method	TSPA Avg (Min, Max)	TSPB Avg (Min, Max)
Random Path	265530.42 (229719, 294157)	213673.99 (195590, 234518)
Local Steepest Edge (Random Path)	74200.79 (71517, 77754)	48421.32 (45631, 52371)
Local Steepest Edge with LM (Random Path)	74123.34 (71525, 78562)	49044.82 (46069, 52197)
MSLS (20 runs x 200 iterations)	71365.90 (70485, 71843)	45905.95 (45389, 46366)
ILS (20 runs, time-limited)	69627.05 (69141, 70453)	43908.50 (43497, 44459)
LNS (20 runs, time-limited, True)	69543.00 (69207, 69904)	43996.75 (43581, 44260)
LNS (20 runs, time-limited, False)	69746.00 (69263, 70633)	44256.45 (43871, 45105)
HEA (20 runs, time-limited, True)	69203.55 (69107, 69383)	43598.45 (43446, 44024)
HEA (20 runs, time-limited, False)	70302.25 (69722, 70901)	44619.30 (44026, 45200)
ILS improved	69121.30 (69095, 69170)	43575.40 (43446, 44086)

Table 3: Comparison of average, minimum, and maximum objective values for TSPA and TSPB. The lowest average values are highlighted in bold.

Method	TSPA Time (avg, min, max) [μ s]	TSPB Time (avg, min, max) [μ s]
Random Path	0.0036 (0.0033, 0.0176)	0.0052 (0.0044, 0.0239)
LS Steepest Edge (Random)	9.7100 (8.1613, 13.9574)	10.2649 (8.0421, 15.3924)
LS Steepest Edge with LM (Random)	9.3588 (6.6432, 12.3063)	9.3000 (7.0456, 12.0311)
MSLS (20 runs x 200 iterations)	1706.4574 (1689.2280, 1791.0236)	1900.8621 (1791.4707, 2033.1554)
ILS (20 runs, time-limited)	1706.6590 (1706.4574, 1707.1652)	1901.0305 (1900.8718, 1901.2089)
LNS (20 runs, time-limited, True)	1709.7860 (1707.4118, 1711.7095)	1903.8503 (1901.1062, 1907.0236)
LNS (20 runs, time-limited, False)	1709.8836 (1706.8554, 1712.9383)	1903.7007 (1901.0532, 1906.1929)
HEA (20 runs, time-limited, True)	1634.6455 (1634.3685, 1636.0765)	1716.7377 (1716.3562, 1717.6336)
HEA (20 runs, time-limited, False)	1634.9237 (1634.3497, 1636.1286)	1716.7424 (1716.3415, 1717.4731)
ILS improved	20004.13 (20000.10, 20012.56)	20005.50 (20000.81, 20014.22)

Table 4: Average, minimum, and maximum execution times for TSPA and TSPB.

5 Conclusion

We strengthened our iterated local search by broadening the starting pool and upgrading the perturbation. Multiple guided starts, paired with one random tour, let us explore diverse regions quickly; the stronger shake breaks stagnation while preserving useful structure. With a time budget focused on the ILS loop, we observed more consistent improvements in solution quality and a clearer picture of performance and runtime under this single, refined method. Overall, these changes make the search more robust and more likely to reach better tours across the tested TSP instances.